

Architecture Evaluation

Achieving Quality Attributes

- Architectural styles provide general beneficial properties. Quality attributes also need to be supported:
 - Modifiability
 - Performance
 - Security
 - Reliability
 - Robustness
 - Usability
 - Business goals

Characteristics that users want to see in products that we build

Achieving Quality Attributes

Modifiability

- Design must be easy to change
- Two classifications of affected software units:
 - Directly affected
 - Indirectly affected
- Directly affected units' responsibilities change to accommodate a system modification
 - Anticipate expected changes
 - Value cohesion
 - Maintain generality
- Indirectly affected units' responsibilities do not change, but implementations must be revised
 - Lower coupling
 - Interact through interfaces
 - Employ multiple interfaces

Cost spent after first version?

Achieving Quality Attributes

Performance

- Performance attributes describe constraints on system speed and capacity:
 - Response time: How fast does our software respond to requests?
 - Throughput: How many requests can it process per minute?
 - Load: How many users can it support before response time and throughput start to suffer?

Achieving Quality Attributes Performance

- Tactics for improving performance include:
 - Improve utilization of resources
 - Manage resource allocation more effectively
 - First-come/first-served: Requests are processed in the order in which they are received
 - Explicit priority: Requests are processed in order of their assigned priorities
 - Earliest deadline first: Requests are processed in order of their impending deadlines
 - Reduce demand for resources

Achieving Quality Attributes

Security

- Two key architectural characteristics particularly relevant to security: immunity and resilience
- **Immunity**: ability to thwart an attempted attack
 - The architecture encourages immunity by:
 - Ensuring all security features are included in the design
 - Minimizing exploitable security weaknesses
- **Resilience**: ability to recover quickly and easily from an attack
 - The architecture encourages resilience by:
 - Segmenting functionality to contain attack
 - Enabling the system to quickly restore functionality

Achieving Quality Attributes

Reliability

- A software system is reliable if it correctly performs its required functions under assumed conditions
 - Is the software internally free of errors?
- A **fault** is the result of human error, compared to a **failure**, which is an observable departure from required behaviour
 - Software is made more reliable by preventing or tolerating faults

Fault= wrong formula

Failure= wrong balance shown

Achieving Quality Attributes

Reliability (continued)

- **Passive fault detection:** wait until fault occurs during execution
- **Active fault detection:** periodically check for symptoms or try to anticipate when failures will occur
- **Exceptions:** situations that cause the system to deviate from its desired behaviour
- Include **exception handling** in design to handle exception and return system to acceptable state

Achieving Quality Attributes

Reliability (continued)

- **Fault recovery:** handling fault immediately to limit damage
- Fault recovery tactics:
 - Undoing transactions: manage a series of actions as a single transaction that are easily undone if a fault occurs midway through the transaction
 - Checkpoint/rollback: software records a checkpoint of current state; rolls back to that point if system gets in trouble
 - Backup: system automatically substitutes faulty unit with backup
 - Degraded service: offers degraded version of the service
 - Correct and continue: detects the problem and treats the symptoms
 - Report: system returns to its previous state and reports the problem to an exception-handling unit

Achieving Quality Attributes

Robustness

- A system is **robust** if it includes mechanisms for accommodating or recovering from problems in the environment or in other unit
- Mutual suspicion: each software unit assumes that the other units contain faults
- Robustness tactics differ from reliability tactics
- Recovery tactics are similar:
 - Rollback to checkpoint state
 - Abort a transaction
 - Initiate a backup unit
 - Provide reduced service
 - Correct symptoms and continue processing
 - Trigger an exception

Achieving Quality Attributes

Usability

- Usability reflects the ease in which a user is able to operate the system
 - User interface should reside in its own software unit
 - Some user-initiated commands require architectural support
 - There are some system-initiated activities for which the system should maintain a model of its environment

Achieving Quality Attributes

Business Goals

- Business Goals are quality attributes the system is expected to exhibit (e.g., minimizing the cost of development and time to market)
 - Buy vs. Build
 - Save development time, money
 - More reliable
 - Existing components create constraints; vulnerable to supplier
 - Initial development vs. maintenance costs
 - Save money by making system modifiable
 - Increased complexity may delay release; lose market to competitors
 - New vs. known technologies
 - Acquiring expertise costs money, delays product release
 - Either learn how to use the new technology or hire new personnel
 - Eventually, we must develop the expertise ourselves

Architecture Evaluation and Refinement

- Design is iterative: we propose design decisions, assess, make adjustments, and propose more decisions
- Many techniques to evaluate the design:
 - Trade-off analysis
 - Cost-benefit analysis
 - Prototyping

Architecture Evaluation and Refinement

Trade-off Analysis

- Often several alternative designs to consider
 - professional duty to explore design alternatives and not simply implement the first design that comes to mind
 - different members of design team may promote competing designs
 - need a measurement-based method for comparing design alternatives

Architecture Evaluation and Refinement

One Specification, Many Designs

- **One specification, many designs**

Architecture Evaluation and Refinement

One Specification, Many Designs (continued)

- Comparison of solutions on the basis of important attributes

Attribute	Arch 1	Arch 2	Arch 3	Arch 4
Easy to change Algorithm	-	-	+	+
Easy to Change Data	-	+	-	-
Easy to Add Functionality	+	-	+	+
Performance	-	-	+	+
Efficient Data Rep	+	+	+	-
Easy to Reuse	-	+	-	+

+ means that the design has the attribute

- means that the attribute is not an aspect of the design

Architecture Evaluation and Refinement

One Specification, Many Designs (continued)

- Weighted comparison of KWIC solutions

Attribute	Priority	Arch 1	Arch 2	Arch 3	Arch 4
Easy to change algorithm	1	1	2	4	5
Easy to change data representation	4	1	5	2	1
Easy to change function	3	4	1	4	5
Good performance	3	5	4	2	2
Easy to reuse	5	1	4	2	5

Highest Priority of an attribute = 5, Lowest priority = 1. For example, Reusability is the most desirable attribute here

Numbers in column 3 to 6 represent the extent to which a design satisfies the characteristic of the corresponding attribute. 1 being the lowest, 5 being the highest

Architecture Evaluation and Refinement

One Specification, Many Designs (continued)

- Weighted comparison of KWIC solutions

Attribute	Priority	Arch 1	Arch 2	Arch 3	Arch 4
Easy to change algorithm	1	1	2	4	5
Easy to change data representation	4	1	5	2	1
Easy to change function	3	4	1	4	5
Good performance	3	5	4	2	2
Easy to reuse	5	1	4	2	5

Multiply the priorities with ratings and sum over the design to get score for each design

For example score for Arch 4= $1 \times 5 + 4 \times 1 + 3 \times 5 + 3 \times 2 + 5 \times 5 = 55$.

Pick the design with the highest score. Here Arch 2 has the highest score (57)

Architecture Evaluation and Refinement

One Specification, Many Designs (continued)

- Other attributes to consider
 - Modularity
 - Testability
 - Security
 - Ease of use
 - Ease of understanding
 - Ease of integration

Architecture Evaluation and Refinement

Cost-Benefit Analysis

- Consider a proposal to improve system's performance
- A cost–benefit analysis is a widely used business tool for estimating and comparing the costs and benefits of a proposed change

Architecture Evaluation and Refinement

Computing Benefits

- A cost-benefit analysis contrasts financial benefits with financial costs, both in Dollars
 - Costs
 - Development
 - Operational
 - Benefits
 - Reduced Operational Costs
 - Increased Earnings
- Payback period
 - the length of time before accumulative benefits recover the costs of implementation

Architecture Evaluation and Refinement

Computing Benefits (Continued)

- Return on Investment (ROI)

- $\text{ROI} = \text{Benefits} / \text{Cost}$, $\text{ROI} > 1$ is desired

- $\% \text{age gain ROI} = (\text{Benefits} - \text{Cost}) / \text{Cost} \times 100$

- Example

- After five years

- Cost = \$ 2249559

- Benefits = \$ 6122893

- $\text{ROI} = 2.72$ i.e. Total benefits are approx. 3 times the total costs

- $\% \text{age gain ROI} = 172.19 \%$ i.e. Your earnings are 172.19 % of the total costs

References

- SE book by Pfleeger
- SE book by Pressman
- SE book by Ian Sommerville
- UCF slides for SE course (a few slides have been reused)